

TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐẠI HỌC QUỐC GIA TP.HCM



BÀI THỰC HÀNH VI XỬ LÝ - VI ĐIỀU KHIỂN

LAB 3

NGUYỄN ĐỖ KHÁNH TRÌNH - 2353237

30/10/2025

Mục lục

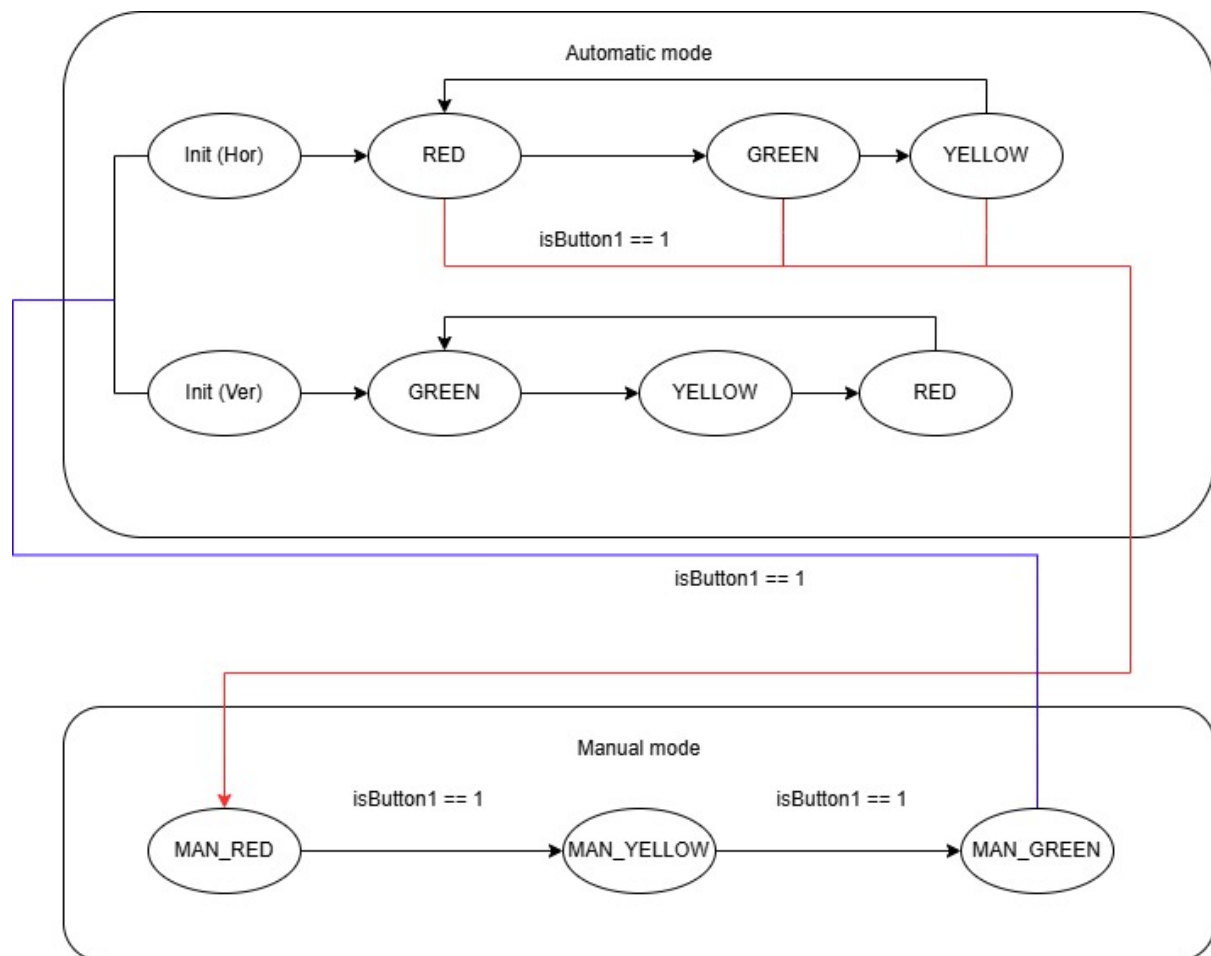
Nội Dung Thực Hành	3
1 FSM sketch:	3
2 Schematic:	4
3 STM32Project with 10ms interrupt:	4
4 Timer parameter setting:	5
5 Exercise 5: Input Reading and Debounce	5
5.1 Mục tiêu	5
5.2 Source Code	5
5.3 Giải thích	6
6 Exercise 6: LED Display and Mode Indication	6
6.1 Mục tiêu	6
6.2 Source Code	6
6.3 Giải thích	7
7 Exercise 7: Adjusting Red Light Duration	7
7.1 Mục tiêu	7
7.2 Giải thích	8
8 Exercise 8: Adjusting Amber Light Duration	8
8.1 Mục tiêu	8
8.2 Giải thích	9
9 Exercise 9: Adjusting Green Light Duration	9
9.1 Mục tiêu	9
9.2 Giải thích	10

Nội Dung Thực Hành

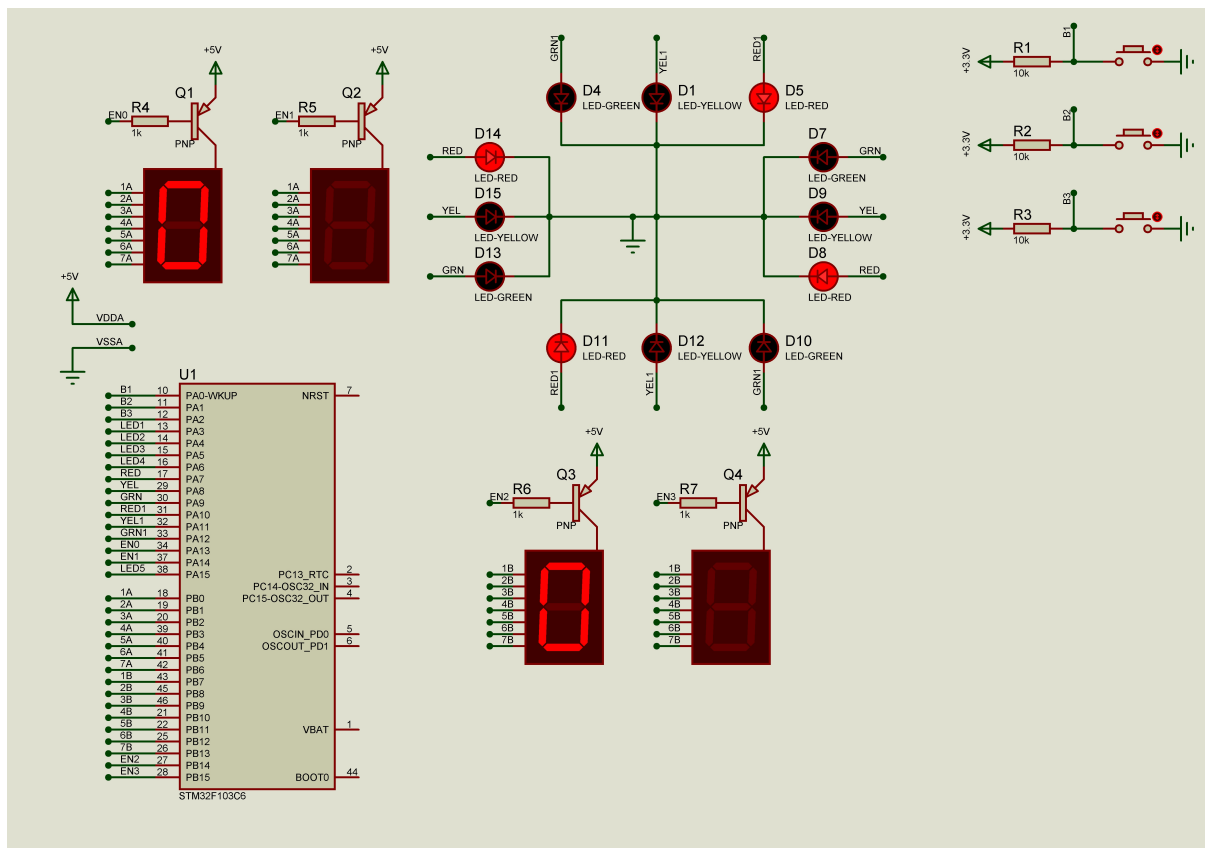
Code và Schematic có thể được xem trực tiếp và tải về tại GitHub Repository: Micro-controller Reports

Demo Lab 3 bằng Video: Link Youtube

1 FSM sketch:

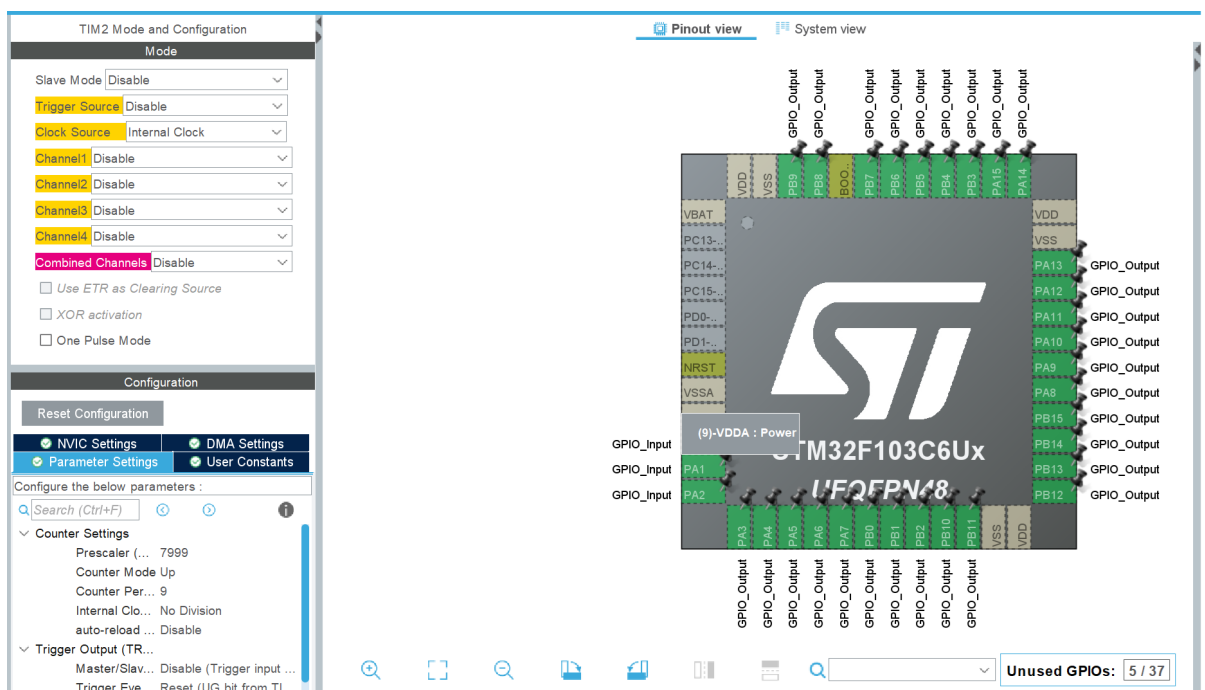


2 Schematic:



3 STM32Project with 10ms interrupt:

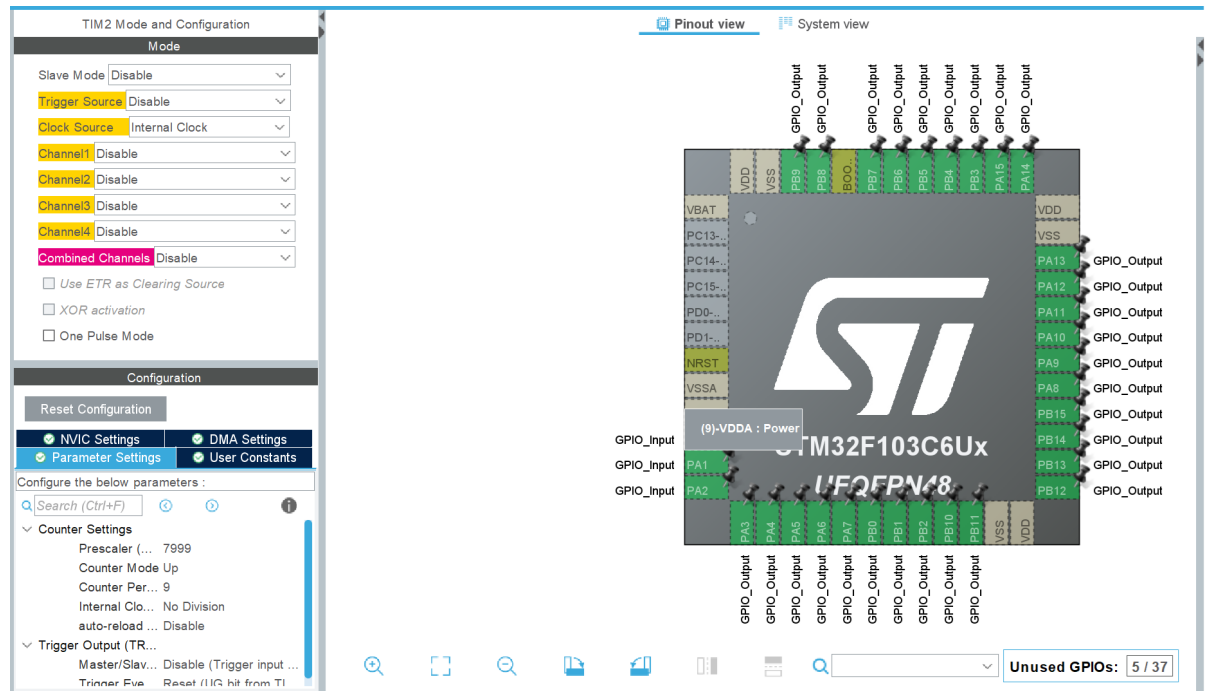
Input frequency của hệ thống là 8MHz.



4 Timer parameter setting:

Input frequency của hệ thống là 8MHz. Chúng ta chỉ cần thay đổi Prescaler và Counter Period thì Timer interrupt sẽ thay đổi.

Ví dụ trong ảnh có Prescaler là 7999 và Counter là 9. Nếu muốn giảm còn 1ms thì có thể tăng Prescaler lên 79999 hoặc giảm còn 799 nếu muốn timer là 100ms. Tương tự với Counter.



5 Exercise 5: Input Reading and Debounce

5.1 Mục tiêu

Thiết kế module `input_reading.c/h` để đọc tín hiệu từ 3 nút nhấn (PA0, PA1, PA2), chống dội phím, và phát hiện nhấn giữ trên 1 giây.

5.2 Source Code

Listing 1: `input_reading.c`

```
#include "input_reading.h"
#include "main.h"

#define N0_OF_BUTTONS 3
#define DURATION_FOR_AUTO_INCREASING 100 // 100 * 10ms = 1s

static GPIO_PinState buttonBuffer[N0_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer1[N0_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer2[N0_OF_BUTTONS];
static uint8_t flagForButtonPress1s[N0_OF_BUTTONS];
```

```

static uint16_t counterForButtonPress1s[N0_OF_BUTTONS];

void button_reading(void){
    for(int i=0; i<N0_OF_BUTTONS; i++){
        debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
        debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0 + i);
        if(debounceButtonBuffer1[i] == debounceButtonBuffer2[i]){
            buttonBuffer[i] = debounceButtonBuffer1[i];
        }

        if(buttonBuffer[i] == GPIO_PIN_RESET){ // pressed
            if(counterForButtonPress1s[i] < DURATION_FOR_AUTO_INCREASING){
                counterForButtonPress1s[i]++;
            } else {
                flagForButtonPress1s[i] = 1;
            }
        } else {
            counterForButtonPress1s[i] = 0;
            flagForButtonPress1s[i] = 0;
        }
    }
}

unsigned char is_button_pressed(uint8_t index){
    if(index >= N0_OF_BUTTONS) return 0;
    return (buttonBuffer[index] == GPIO_PIN_RESET);
}

unsigned char is_button_pressed_1s(unsigned char index){
    if(index >= N0_OF_BUTTONS) return 0;
    return flagForButtonPress1s[index];
}

```

5.3 Giải thích

Hàm `button_reading()` được gọi mỗi 10 ms trong ngắt TIM2. Bộ đệm 2 mẫu liên tiếp được so sánh để loại bỏ nhiễu; nếu giữ nút > 1 giây, cờ `flagForButtonPress1s` sẽ bật.

6 Exercise 6: LED Display and Mode Indication

6.1 Mục tiêu

Hiển thị số đếm và chế độ hiện tại trên LED 7 đoạn, 4 LED đơn để báo Mode 1 – 4.

6.2 Source Code

Listing 2: `led_display.c`

```

#include "led_display.h"
#include "main.h"

void led_mode_display(int index){
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);

    switch(index){
        case 1: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET); break;
        case 2: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET); break;
        case 3: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); break;
        case 4: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET); break;
        default: break;
    }
}

```

6.3 Giải thích

Bốn LED PA3–PA6 bật luân phiên để báo mode hiện hành (1→4). Các LED 7 đoạn được quét ở module `input_processing.c` bằng 2 bộ đệm 2 digit cho mỗi hướng giao thông.

7 Exercise 7: Adjusting Red Light Duration

7.1 Mục tiêu

Ở Mode 2, nhấn BTN2 để tăng thời gian đèn đỏ, BTN3 giữ 1 giây để lưu.

Source Code

Listing 3: `input_processing.c` (Mode 2 – RED)

```

if (mode == 2){
    if (timer2_flag == 1){
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_7 | GPIO_PIN_10); // blink RED L
        setTimer2(500);
    }

    update_led_1_buffer(temp_duration[red]);
    if (timer3_flag == 1){
        update7SEG_led_1(index_led_1);
        index_led_1 = (index_led_1 + 1) % 2;
        update_led_2_buffer(temp_duration[red]);
        update7SEG_led_2(index_led_2);
        index_led_2 = (index_led_2 + 1) % 2;
        setTimer3(200);
    }
}

```

```

    fsm_change_duration(red);    // BTN2: increase
    fsm_set_duration(red);      // BTN3: save after hold
}

```

Listing 4: Hàm thay đổi và lưu thời gian

```

void fsm_change_duration(int color){
    if(is_button_pressed(1)){
        if (temp_duration[color] < 99) temp_duration[color]++;
        else temp_duration[color] = 1;
    }
    if(is_button_pressed_1s(1)){
        if(timer4_flag == 1){
            temp_duration[color]++;
            if(temp_duration[color] > 99) temp_duration[color] = 1;
            setTimer4(100);
        }
    }
}

void fsm_set_duration(int color){
    if(is_button_pressed_1s(2)){
        color_duration[color] = temp_duration[color];
    }
}

```

7.2 Giải thích

Khi vào Mode 2, các LED đỏ nháy 2 Hz; BTN2 tăng giá trị vòng (1–99), BTN3 giữ > 1 giây để ghi vào mảng `color_duration[]`.

8 Exercise 8: Adjusting Amber Light Duration

8.1 Mục tiêu

Ở Mode 3, chỉnh thời gian đèn vàng tương tự Mode 2.

Source Code

Listing 5: input_processing.c (Mode 3 – AMBER)

```

if (mode == 3){
    if (timer2_flag == 1){
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8 | GPIO_PIN_11); // blink YELLOW
        setTimer2(500);
    }
}

```



```

    update_led_1_buffer(temp_duration[amber]);
    if (timer3_flag == 1){
        update7SEG_led_1(index_led_1);
        index_led_1 = (index_led_1 + 1) % 2;
        update_led_2_buffer(temp_duration[amber]);
        update7SEG_led_2(index_led_2);
        index_led_2 = (index_led_2 + 1) % 2;
        setTimer3(200);
    }

    fsm_change_duration(amber);
    fsm_set_duration(amber);
}

```

8.2 Giải thích

LED vàng nhấp nháy 2 Hz khi chỉnh. BTN2 tăng thời gian, BTN3 giữ 1 giây để lưu giá trị vào `color_duration[amber]`.

9 Exercise 9: Adjusting Green Light Duration

9.1 Mục tiêu

Ở Mode 4, chỉnh thời gian đèn xanh theo cách tương tự.

Source Code

Listing 6: `input_processing.c` (Mode 4 – GREEN)

```

if (mode == 4){
    if (timer2_flag == 1){
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_9 | GPIO_PIN_12); // blink GREEN
        setTimer2(500);
    }

    update_led_1_buffer(temp_duration[green]);
    if (timer3_flag == 1){
        update7SEG_led_1(index_led_1);
        index_led_1 = (index_led_1 + 1) % 2;
        update_led_2_buffer(temp_duration[green]);
        update7SEG_led_2(index_led_2);
        index_led_2 = (index_led_2 + 1) % 2;
        setTimer3(200);
    }

    fsm_change_duration(green);
    fsm_set_duration(green);
}

```

9.2 Giải thích

Các LED xanh nháy 2 Hz để báo hiệu đang chỉnh thời gian đèn xanh. BTN2 tăng giá trị, BTN3 giữ 1 giây để xác nhận và lưu lại.